

Mind Care — Student Mental Wellness Portal

Project Overview: Mind Care is a student-focused web application designed to help users track their daily mood and stress, maintain a private journal, and access simple wellness resources. The system provides non-clinical wellness indicators and self-care tools; it is not intended to diagnose or treat medical or mental-health conditions.

Recommended feature set

Core features

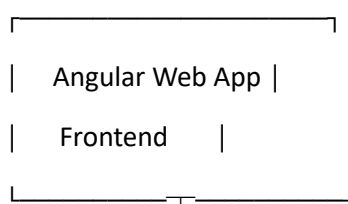
1. Student registration/login
2. Student profile
3. Daily mood tracking
4. Daily stress assessment
5. Personal journal
6. Mood/stress history
7. Dashboard with charts
8. Personalized wellness tips
9. Meditation/breathing exercises
10. Sleep tracking
11. Habit/wellness tracking
12. Wellness goals
13. Notifications/reminders
14. Mental-health resource library
15. Emergency/help resources
16. Admin panel for managing wellness content

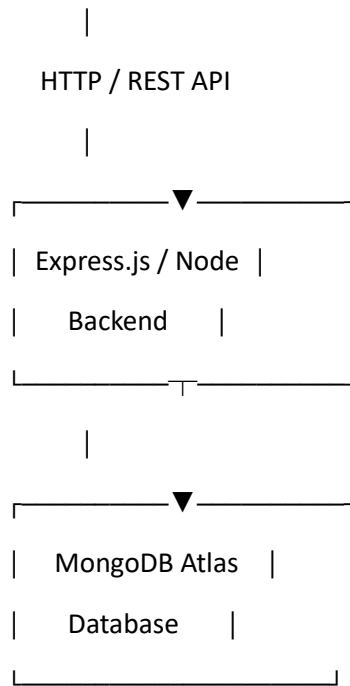
Optional advanced feature

17. **AI Wellness Assistant** — students can ask general wellness/study-stress questions and receive supportive suggestions. It should not diagnose mental-health conditions.

1. How the system should work

A simple architecture would be:





If you add AI:

Angular



Express.js



AI API



Response



Angular

2. Pages to create

I recommend approximately **12–15 pages**.

Public pages

/

└— Home

└— About

└— Wellness Resources

└— Login

└─ Register

Student pages

/dashboard

/mood

/stress-assessment

/journal

/analytics

/wellness

/meditation

/sleep

/habits

/goals

/profile

Admin

/admin

/admin/users

/admin/resources

/admin/meditations

/admin/tips

You don't necessarily need to build every admin page initially.

3. Main Dashboard

This should be the most important page.

Example:

Good Morning, Jitesh 🙌	
How are you feeling today?	
😄 😊 😐 😞 😓	

Today's Stress	Mood
Low	Good
Mood this week	
 Chart	
Recommended for you	
 5-minute breathing exercise	
 Take a study break	
 Improve your sleep routine	

4. Mood Tracker

Create a simple form.

How are you feeling today?

 Very Good

 Good

 Okay

 Bad

 Very Bad

What affected your mood?

- ☐ Studies
- ☐ Exams
- ☐ Family
- ☐ Friends
- ☐ Health
- ☐ Finance
- ☐ Personal
- ☐ Other

Additional note:

[_____]

[Save Mood]

Store the result in MongoDB.

MongoDB document

```
{
  userId: "...",
  mood: 4,
  reasons: ["Studies", "Exams"],
  note: "Feeling nervous about exams",
  date: "2026-09-11"
}
```

5. Stress Assessment

This is one of the most important parts of your **research project**.

Create around **10 questions**.

Example:

How often have you felt overwhelmed by your studies?

Never 0

Sometimes 1

Often 2

Very Often 3

After the questionnaire:

Your Stress Level

 Low Stress

Score: 7 / 30

Recommended:

- Take regular study breaks
- Try 5-minute breathing
- Maintain a regular sleep schedule

You can define:

0–9 → Low

10–19 → Moderate

20–30 → High

These thresholds are **application rules**, not a clinical diagnosis.

6. Journal

Students can write:

Today's Journal

Title:

[My day]

How are you feeling?

[_____]

Write your thoughts:

[]

[]

[]

Mood:



[Save Journal]

Features:

- Create journal
- Edit journal
- Delete journal
- View previous entries
- Search entries
- Filter by date

MongoDB

```
{  
  userId: "...",  
  title: "My day",  
  content: "...",  
  mood: 3,  
  createdAt: Date  
}
```

7. 📊 Mood & Stress Analytics

This would make your project much stronger.

Show:

Weekly mood

Mon 😐 4

Tue 😐 3

Wed 😐 2

Thu 😊 4

Fri 😄 5

Sat 😄 5

Sun 😊 4

Then display a graph.

You can use:

- Chart.js
- ng2-charts
- another Angular-compatible chart library

Show:

Average Mood

Average Stress

Highest Stress Day

Lowest Mood Day

Weekly Progress

This also gives you something meaningful to discuss in the **research/project report**.

8. 🧘 Wellness Recommendations

Don't just randomly display tips.

Make them dependent on the user's data.

For example:

IF stress < 10

→ Show general wellness tips

IF stress >= 10 AND stress < 20

→ Show relaxation + study break recommendations

IF stress >= 20

→ Show stronger wellness/support recommendations

For example:

Stress: Moderate

Recommended:

-  Breathing Exercise
-  Take a 10-minute study break
-  Drink some water
-  Take a short walk
-  Maintain a regular sleep schedule

9. **Meditation Section**

Create a library of exercises.

Example:

Meditation & Relaxation

5-Minute Breathing	
Beginner	
⌚ 5 min	
[Start]	

Stress Relief	
Beginner	
⌚ 10 min	
[Start]	

Sleep Relaxation	

| Beginner |

| ⌚ 10 min |

| [Start] |

You could initially implement a simple **breathing timer** rather than depending entirely on external videos.

Example:

Breathe In

4 sec

Hold

4 sec

Breathe Out

6 sec

10. 🧘 Sleep Tracker

Keep it simple.

Sleep Tracker

Date: 11/09/2026

Sleep time:

[11:30 PM]

Wake-up time:

[07:00 AM]

Sleep quality:

★ ★ ★ ★ ☆

[Save]

Then show:

Average Sleep

7h 12m

Sleep Quality

Good

11. 🎯 Wellness Goals

Students can create goals.

Example:

My Goals

☒ Meditate for 10 minutes

5 / 7 days

☐ Sleep before 11:30 PM

3 / 7 days

☒ Take study breaks

6 / 7 days

This gives your application a **progress tracking** component.

12. 🔔 Reminders

Add reminders such as:

🕒 Daily Mood Check-in

🕒 Journal Reminder

🕒 Meditation Reminder

🕒 Take a Study Break

For a student project, you can initially implement browser notifications or in-app reminders rather than building a complex notification infrastructure.

13. Help & Resources

This is important for a mental-wellness application.

Create:

Need Support?

I'm feeling stressed



Wellness resources

I'm feeling overwhelmed



Talk to someone

I need professional help



Counselling / professional resources

Include appropriate **local emergency and professional-support information** rather than trying to make the application itself a crisis service.

Also display a disclaimer such as:

Mind Care provides general wellness information and self-care tools. It does not provide medical diagnosis or replace professional mental-health care.

14. Admin Panel

This makes the project feel much more like a real web application.

Admin can:

- View users
- Add wellness tips
- Edit tips
- Delete tips

- Add meditation resources
- Manage articles
- View application statistics

For example:

Admin Dashboard

Total Students	245
Mood Entries	1,832
Journal Entries	932
Assessments	1,205

Most Common Mood 🙄 Okay

Average Stress Moderate

Important: Avoid giving administrators access to private journal contents unless your project explicitly requires it and has a strong privacy justification. For a student wellness app, keeping journals private is the better design.

15. 🤖 Optional AI Assistant

If you want an advanced feature, add:

"Mind Care Assistant"

Student:

"I'm feeling stressed because I have three assignments."

AI:

"It sounds like you have a lot on your plate. You could try breaking the assignments into smaller tasks and taking short breaks between study sessions."

It can provide:

- General wellness suggestions
- Study-stress management
- Relaxation suggestions
- Journaling prompts
- Meditation recommendations

But **don't make it diagnose depression, anxiety, etc.**

16. MongoDB Collections

I'd structure your database approximately like this:

mindcare

|

├─ users

|

├─ moodEntries

|

├─ stressAssessments

|

├─ journals

|

├─ sleepEntries

|

├─ habits

|

├─ goals

|

├─ wellnessTips

|

├─ meditationResources

|

├─ notifications

|

└─ aiChats ← optional

Users

{

 name,

 email,

 password,

```
  role,  
  createdAt  
}
```

Mood

```
{  
  userId,  
  mood,  
  reasons,  
  note,  
  date  
}
```

Stress Assessment

```
{  
  userId,  
  answers,  
  score,  
  level,  
  recommendations,  
  createdAt  
}
```

Journal

```
{  
  userId,  
  title,  
  content,  
  mood,  
  createdAt,  
  updatedAt  
}
```

17. Backend structure

Your Express backend can look like:

backend/

|

├── config/

| └─ db.js

|

├── controllers/

| └─ authController.js

| └─ moodController.js

| └─ stressController.js

| └─ journalController.js

| └─ sleepController.js

| └─ wellnessController.js

|

├── models/

| └─ User.js

| └─ Mood.js

| └─ StressAssessment.js

| └─ Journal.js

| └─ Sleep.js

| └─ WellnessTip.js

|

├── routes/

| └─ authRoutes.js

| └─ moodRoutes.js

| └─ stressRoutes.js

| └─ journalRoutes.js

| └─ wellnessRoutes.js

|

└─ middleware/

```
|   ├── authMiddleware.js
|   └── adminMiddleware.js
|
|   ├── .env
|   ├── server.js
|   └── package.json
```

18. Angular structure

frontend/

```
└── src/
    ├── app/
    |   |
    |   ├── core/
    |   |   ├── services/
    |   |   └── guards/
    |   |
    |   ├── auth/
    |   |   ├── login/
    |   |   └── register/
    |   |
    |   ├── dashboard/
    |   |
    |   ├── mood/
    |   |
    |   ├── stress-assessment/
    |   |
    |   ├── journal/
    |   |
    |   └── analytics/
    |
    |
```

- └─ wellness/
- |
- └─ meditation/
- |
- └─ sleep/
- |
- └─ goals/
- |
- └─ profile/
- |
- └─ admin/

19. What to put in .env

For the **Express/Node backend**, something like:

PORT=5000

NODE_ENV=development

MONGODB_URI=mongodb+srv://USERNAME:PASSWORD@cluster0.xxxxx.mongodb.net/mindcare?retryWrites=true&w=majority

JWT_SECRET=your_long_random_secret_key

JWT_EXPIRES_IN=7d

CLIENT_URL=http://localhost:4200

If you add email:

EMAIL_HOST=smtp.gmail.com

EMAIL_PORT=587

EMAIL_USER=your_email@gmail.com

EMAIL_PASSWORD=your_app_password

If you add AI:

GEMINI_API_KEY=your_gemini_api_key

So your .env could eventually be:

PORT=5000

NODE_ENV=development

MONGODB_URI=mongodb+srv://USERNAME:PASSWORD@cluster0.xxxxx.mongodb.net/mindcare?retryWrites=true&w=majority

JWT_SECRET=replace_with_a_long_random_secret

JWT_EXPIRES_IN=7d

CLIENT_URL=http://localhost:4200

GEMINI_API_KEY=your_gemini_api_key

EMAIL_HOST=smtp.gmail.com

EMAIL_PORT=587

EMAIL_USER=your_email@gmail.com

EMAIL_PASSWORD=your_gmail_app_password

 **Very important**

Do **not** put:

MONGODB_URI

JWT_SECRET

GEMINI_API_KEY

EMAIL_PASSWORD

inside your Angular frontend.

And don't upload .env to GitHub.

Add:

.env

.env.*

!.env.example

Then create:

.env.example

with placeholders:

PORT=5000

NODE_ENV=development

MONGODB_URI=

JWT_SECRET=

JWT_EXPIRES_IN=7d

CLIENT_URL=http://localhost:4200

GEMINI_API_KEY=

EMAIL_HOST=

EMAIL_PORT=587

EMAIL_USER=

EMAIL_PASSWORD=

20. API structure

Your frontend can communicate with Express through APIs like:

Authentication

POST /api/auth/register

POST /api/auth/login

GET /api/auth/me

PUT /api/auth/profile

Mood

POST /api/mood

GET /api/mood

GET /api/mood/weekly

DELETE /api/mood/:id

Stress

POST /api/stress/assessment

GET /api/stress/history

GET /api/stress/latest

Journal

POST /api/journal

GET /api/journal

GET /api/journal/:id

PUT /api/journal/:id

DELETE /api/journal/:id

Wellness

GET /api/wellness/tips

GET /api/wellness/meditation

Sleep

POST /api/sleep

GET /api/sleep

21. Recommended development plan

Don't try to build everything simultaneously.

Phase 1 — Project setup

First create:

Angular frontend

+

Express backend

+

MongoDB Atlas

Get this working first.

Phase 2 — Authentication

Build:

Register

↓

Login

↓

JWT



Protected Dashboard

Test this before continuing.

Phase 3 — Core wellness features

Build in this order:

1. Mood Tracker



2. Stress Assessment



3. Journal



4. Dashboard

At this point you already have a working project.

Phase 4 — Analytics

Add:

Mood history

Stress history

Weekly charts

Monthly charts

Average mood

Average stress

Phase 5 — Wellness

Add:

Wellness tips

Meditation

Breathing exercise

Sleep tracking

Habit tracking

Goals

Phase 6 — Admin

Build:

Admin Login



Admin Dashboard



Manage Tips

Manage Resources

Manage Meditation Content

View Statistics

Phase 7 — Optional AI

Only after everything above works:

Student



Mind Care Assistant



Express Backend



Gemini API



Response

Suggested technology stack

Frontend

Angular 20+

Backend

Node.js

Express.js

Database

MongoDB Atlas

Authentication

JWT + bcrypt

AI (optional)

Gemini API

Deployment

Vercel → Angular

Render → Express

MongoDB Atlas → Database